https://github.com/ritesh2687/dsa_ppt_post

DAY 81

Building a Binary Tree from Input in C++

From User Input to Dynamic Tree Construction using Recursion & Sentinel Values

Continuously building on Day 80: Level Order Traversal

C++17 Edition

Learning Journey: Day 77 → Day 81

https://github.com/ritesh2687/dsa_ppt_post

DAY 77-78

Node Structure & Traversals

Learned node pointers, left/right child connections, Preorder, Inorder, Postorder.

DAY 79-80

Advanced Traversals & Queues

Mastered stack-based iterative traversals and level-order traversal using queues.

TODAY: DAY 81

Dynamic Tree Construction

Building full binary trees dynamically from user input streams in C++17.

Previous knowledge assumed: Recursion, pointer manipulation, queue operations.

Why Build a Tree from Input?

Hardcoded Tree Limitation

Manually wiring pointers (`root->left = new Node(20)`) is rigid, unscalable, and impractical for dynamic datasets.

Dynamic Input Advantage

Accepting input streams allows programs to construct arbitrary tree shapes dynamically at runtime based on user data.

Real-World Use Cases

- ✓ Parsing Abstract Syntax Trees (AST) in compilers
- ✓ Reconstructing trees from serialized file inputs
- ✓ Decision trees in machine learning pipelines

Understanding the Input Format

Preorder with Null Markers

We provide values in Preorder sequence (Root → Left → Right). When a node has no child, we explicitly represent it using a **sentinel value**.

```
Sentinel Value: -1 (Represents `nullptr`)
```

Rule 1: Node Value

If value != -1, create a new node and proceed.

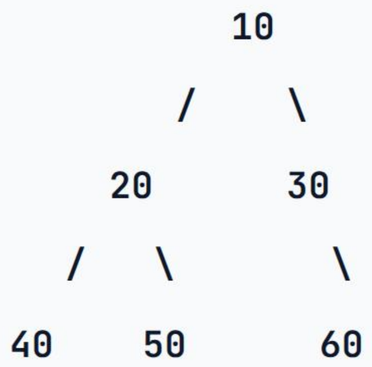
Rule 2: Null Marker

If value == -1, return `nullptr` and stop current branch.

Rule 3: Constraint

-1 is reserved and cannot be a valid node data value.

Sample Tree to Construct



Tree Specifications

- **Root Node:** 10
- **Left Subtree:** Root 20 with children 40 and 50
- **Right Subtree:** Root 30 with right child 60 (left child null)

We will use this exact tree structure consistently across all code examples and dry runs.

The Input Sequence

PREORDER INPUT STREAM WITH NULL MARKERS

```
10   20   40   -1   -1   50   -1   -1   30   -1   60   -1   -1
```

How it maps:

Root (10) → Left (20) → Left of 20 (40) → Nulls for 40 → Right of 20 (50) → Nulls for 50 → Right (30) → Null left → Right of 30 (60) → Nulls for 60.

Why Preorder?

Preorder processes root first, making it straightforward to instantiate the parent node before recursing into its children.

Exact token stream fed into `cin >> value`.

Node Structure Recap


```cpp
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};
```

Constructor Initialization

Automatically sets `left` and `right` pointers to `nullptr` upon node creation.

Foundation from Day 77

No changes needed to the core Node structure; it natively supports our dynamic build function.

Standard C++17 struct blueprint.

Recursive Construction Intuition

1. Read Root

Read next integer from stream. If -1, return `nullptr`. Otherwise, instantiate current node.

2. Build Left

Recursively call `buildTree()` and assign result to `root->left`. Recursion handles depth automatically.

3. Build Right

Recursively call `buildTree()` and assign result to `root->right`. Return completed `root` pointer.

Divide and conquer applied to tree building.

Base Case: Empty Node


```
int value;
cin >> value;

if (value == -1) {
    return nullptr;
}
```

Stopping Recursion

The base case prevents infinite recursion by detecting the sentinel value `-1` and safely returning `nullptr`.

Pointer Safety

Ensures child pointers of leaf nodes remain cleanly terminated as `nullptr`.

Recursive Build Function


```
Node* buildTree() {
    int value;
    cin >> value;

    if (value == -1) return nullptr;

    Node* root = new Node(value);
    root->left = buildTree();
    root->right = buildTree();
    return root;
}
```

1. Read & Check
Reads token, checks if -1.

2. Allocate Root
Creates node with current value.

3. Recurse Subtrees
Builds left then right subtree sequentially.

Concise core recursive implementation.

Full C++17 Program

https://github.com/ritesh2687/dsa_ppt_post

```
#include
using namespace std;

struct Node {
    int data; Node* left; Node* right;
    Node(int v) : data(v), left(nullptr), right(nullptr) {}
};

Node* buildTree() {
    int val; cin >> val;
    if (val == -1) return nullptr;
    Node* root = new Node(val);
    root→left = buildTree();
    root→right = buildTree();
    return root;
}
int main() {
    Node* root = buildTree();
    // Verification logic here
    return 0;
}
```

Compilable & Clean

This complete program successfully reads the preorder token stream and instantiates the entire tree structure in memory.

`Tested with C++17 standard flags.`

Dry Run: Root Node Creation

STEP 1

Reading First Token: 10

Input stream gives `10`. Since `10 != -1`, we allocate `Node* root = new Node(10)`.

```
Root Pointer = 0x7ffd... (Node with data 10)
```

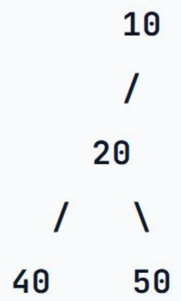


Root Established

Dry Run: Left Subtree (20, 40, 50)

Execution Trace

- Next token `20` becomes `root->left`.
- Token `40` becomes left child of 20.
- Tokens `-1 -1` assign `nullptr` to 40's children.
- Token `50` becomes right child of 20, with `-1 -1` children.

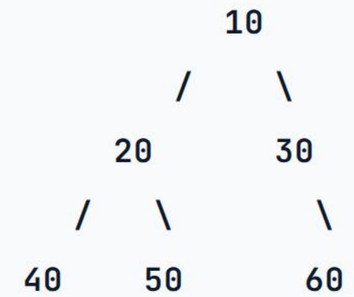


Recursive descent constructing the left branch.

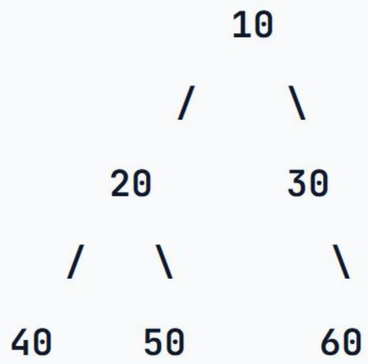
Dry Run: Right Subtree (30, 60)

Completing Construction

- Token `30` becomes `root->right`.
- Token `-1` assigns `nullptr` to 30's left child.
- Token `60` becomes right child of 30.
- Final `-1 -1` terminate 60's children.



Final Constructed Tree Overview



Memory Integrity Confirmed

All pointers successfully established. Zero memory leaks when paired with proper destructor/cleanup logic.

```
Structure matches intended user input perfectly.
```

Verify the Constructed Tree

https://github.com/ritesh2687/dsa_ppt_post

PREORDER

10 20 40 50 30 60

Matches original input sequence without null markers.

INORDER

40 20 50 10 30 60

Validates left-root-right symmetric traversal order.

LEVEL ORDER (DAY 80)

10 20 30 40 50 60

Validates breadth-first queue traversal ordering.

Using previous traversal functions purely for verification.

Alternative: Level-Order Input Construction

Queue-Based Building

Instead of preorder recursion, trees can also be constructed iteratively using a queue from level-order input streams.

```
Connects left and right children per parent popped from queue.
```

Key Difference

Recursive preorder builds depth-first using the call stack, whereas queue-based building constructs level by breadth.

Common Mistakes & Pitfalls

1. Missing Base Case

Forgetting `if (val == -1) return nullptr` causes stack overflow.

2. Wrong Call Order

Swapping left and right recursive assignment order mismatches input.

3. Sentinel Collision

Using -1 as valid node data when it is reserved for null markers.

Interactive Construction Challenge

Challenge Input Stream

```
1   2   -1   -1   3   -1   -1
```

Draw the resulting tree and predict its Preorder & Inorder outputs.

Answer Reveal

- **Root:** 1 with left child 2 and right child 3.
- **Preorder:** 1 2 3
- **Inorder:** 2 1 3

Revision & Day 82 Preview

https://github.com/ritesh2687/dsa_ppt_post

TODAY RECAP

Recursive Tree Construction

Mastered sentinel null markers (-1) and recursive preorder stream allocation in C++17.

TOMORROW: DAY 82

Construct Tree from Preorder & Inorder

Learn how traversal sequences alone can uniquely reconstruct binary tree structures.

Star the repository to support the series!